第1课

从 Python 上手 C++ 基础

第一课 · S01–S80

AIM Robotics

2026–2027 新人培训 · C++17

适合已经会 Python，刚开始学习 C++ 的同学。先建立程序运行、CMake 本地构建、流和类型的认识，再学习容器、函数、内存、引用与指针。

本讲义包含概念介绍和本课全部练习题面。代码解释片段用于说明当前概念；题目“初始代码”需要独立补全后运行。

阅读顺序

1.1 程序、编译与输入输出流 3

1.2 类型、变量与算术 9

1.3 条件判断与输入状态 16

1.4 循环、EOF 与逐步变化的状态 22

1.5 数组、vector 与一批数据 30

1.6 字符串、整行输入与字符扫描 38

1.7 字符串流与严格解析 45

1.8 函数、作用域与值传递 50

1.9 内存、地址与引用 56

1.10 指针、区间与对象有效期 60

1.11 结构体、枚举与记录集合 65

练习与阅读

每个小节先介绍概念，再列出对应练习。题号沿用 CPP-Classroom，题目按本课的学习顺序出现。样例没有输入时标为“无输入”，其含义与平台上的空样例输入相同。

概念代码片段只展示当前需要解释的部分。运行某道题时，以该题完整初始代码为起点，保留所需头文件与唯一的 main。讲义不在题面前附上整题参考答案。

1.1 程序、编译与输入输出流

你已经会写 Python 程序。进入 C++ 的第一步，是弄清源文件怎样成为正在运行的程序，以及数据怎样进出程序。

本节题号：S01、S02、S07、S08

源代码需要一种执行方式

源代码是人写的文本，处理器实际执行的是机器指令。常见 C++ 工具链先预处理源文件，再编译和汇编，得到目标文件，随后由链接器把所需定义和库连接成可执行程序。初学时可以先记住“构建成功，然后运行”这条主线。编译器负责检查语法和类型等规则，链接器还要为被调用的函数找到定义。一次点击运行可能同时触发多个构建步骤。

常见 Python 实现 CPython 也有编译步骤：源码通常先成为字节码，再交给解释器执行。我们平时直接运行 .py，往往看不到独立的构建命令。“编译型与解释型”适合用于比较常见使用方式，但不能简化成“Python 从不编译”或者“解释器只会逐行翻译”。这门课使用 C++17，平台接收一份完整的 .cpp 源文件，每次运行都重新开始程序。

常见工作流程	学生能观察到的过程
CPython	源代码编译为字节码，由解释器执行
C++ 工具链	源代码经构建形成可执行程序，再启动运行
运行失败时	先确定错误发生在构建阶段还是程序运行之后

读懂一份完整程序

```
#include <iostream>

int main() {
    std::cout << "Hello, C++!" << '\n';
    return 0;
}
```

以 # 开头的 include 是预处理指令，头文件让当前源文件看到需要的接口声明。main 是这里的程序入口，函数体由大括号围住，多数语句以分号结束。int 表示 main 返回整数退出状态。return 0 表示正常结束，它不会把数字 0 打印到输出区。输出内容和退出状态是两条不同的信息。

流连接程序和数据

把“流”先理解为程序连续读取或写入字符数据的接口。输入流把外部字符交给程序，输出流接收程序写出的字符。标准输入常连接终端或平台准备的输入，标准输出常连接终端或平台输出区，也可以重定向到文件，因此输入流不能直接等同于键盘，输出流也不能直接等同于屏幕。

名称	身份	初学时的用途
<iostream>	头文件	提供标准流对象的声明及所需接口
std::istream / std::ostream	类型	分别表示输入流与输出流
std::cin	标准输入流对象	读取输入
std::cout	标准输出流对象	写出正常结果
std::cerr	标准错误流对象	写出诊断信息

std::cin 是 std::istream 类型的对象，std::cout 和 std::cerr 是 std::ostream 类型的对象。另一个类型名 std::iostream 表示同时支持输入与输出的流，要与 <iostream> 这个头文件名区分。std::cout << value 把值的文本表示插入输出流，连续使用 << 可以拼接多个字段。\n 是换行字符，std::endl 则在换行之外还会刷新输出缓冲。通常用 \n 表达普通换行。缓冲是暂存待处理字符的区域，不必要求每次写入立即出现在物理设备上。先学会接口的可观察行为，后面再看输入位置和失败状态。

配套练习

S01 Hello 与退出码

源文件：S01.cpp

输出两行 AIM 和 C++17，正常结束程序。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
无输入	AIM C++17

初始代码

```
#include <iostream>

int main() {
    // TODO
    return 0;
}
```

S02 多个字段输出

源文件：S02.cpp

把机器人编号 7 与名称 Sentry 输出成 id=7 name=Sentry。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
无输入	`id=7 name=Sentry`

初始代码

```
#include <iostream>
#include <string>

int main() {
    // TODO
    return 0;
}
```

用 CMake 描述一个本地项目

平台替你安排了单文件的编译和运行；本地项目则需要说明要构建什么、哪些源文件参与构建、采用什么语言标准。CMake 用 CMakeLists.txt 描述这些信息，并为本机生成合适的构建系统。构建工具再组织编译器和链接器完成工作。CMake、构建工具与编译器各有职责，安装 CMake 并不会自动代替 C++ 编译器。

工具	本次构建中的职责
CMake	读取项目描述、检查工具链并生成构建系统
Make、Ninja 或 IDE 构建工具	按构建规则调用编译和链接步骤
Clang、GCC 或 MSVC	把 C++ 源代码编译、链接成程序

这里沿用已经完成的 S01，不新增题目。在本地创建一个文件夹，把 S01 的完整程序保存为 S01.cpp，再在同一目录新建 CMakeLists.txt，注意文件名的大小写和 .txt 后缀。源码目录保存你维护的文件，build 目录保存生成的构建文件与程序，两者分开，便于找到自己的代码。开始前在本地终端运行 cmake --version，确认命令可用，并准备好 C++ 编译器和相应构建工具。

```
cpp-start/
    CMakeLists.txt
    S01.cpp
    build/
```

读懂最小 CMakeLists.txt

```
cmake_minimum_required(VERSION 3.16)
project(aim_cpp LANGUAGES CXX)
set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)
add_executable(S01 S01.cpp)
```

第一行要求使用至少 3.16 的 CMake，这个版本号与 C++17 的语言版本不是一回事。project 为项目命名，并用 CXX 表示项目需要 C++。三个 set 分别请求 C++17、要求编译器支持所请求的标准，并选择不使用编译器扩展的标准模式。它们放在目标创建之前，为后面创建的目标设置相应默认属性。

add_executable 声明一个可执行目标。第一个 S01 是目标名，后面的 S01.cpp 是参与构建的源文件名；项目名 aim_cpp、目标名 S01 与文件名 S01.cpp 分别承担不同职责。这里一个目标只选择一份带 main 的练习程序，不能把多道各有 main 的题随意拼进同一个目标。

配置、构建、运行是三个步骤

在包含 CMakeLists.txt 的目录中打开终端。先配置项目，-S . 指定当前目录为源码目录，-B build 指定生成文件放到 build 目录。配置成功说明项目描述和基本工具链检查通过，不等于你的程序已经编译成功。随后执行构建命令，才会真正编译和链接源文件。

```
cmake –S . –B build
cmake ––build build
```

构建完成后，还需要启动生成的程序。在 macOS 或 Linux 常见的单配置构建中，可执行文件直接位于 build 中；Windows 单配置构建通常会加上 .exe 后缀。使用 Visual Studio 等多配置生成器时，可以执行 cmake --build build --config Debug，再到 build 的 Debug 子目录运行。下表列出当前最小项目的常见命令；实际路径取决于操作系统、生成器和构建配置。

常见环境	构建后的运行命令
macOS / Linux 单配置	./build/S01
Windows 单配置	.\build\S01.exe
Windows 多配置 Debug	.\build\Debug\S01.exe

修改 S01.cpp 并保存后，再执行 cmake --build build，然后重新运行程序。修改 CMakeLists.txt 后可以重新执行配置，再进行构建；一些生成器也会在构建时自动触发重新配置。若使用单配置生成器并希望启用 Debug，可在配置时额外传入 -DCMAKE_BUILD_TYPE=Debug；多配置生成器则在构建时用 --config Debug 选择配置。先掌握当前这一个目标的流程即可。

把构建错误放回相应阶段

找不到 CMakeLists.txt 时检查源码目录和文件名；找不到编译器时检查本机工具链；编译或链接失败时查看 C++ 诊断；构建成功却找不到程序时检查实际输出路径和配置。终端中的 cmake 命令属于本地工具操作，不写进 C++ 源文件。CPP-Classroom 的课堂练习继续提交单份 .cpp，CMakeLists.txt 用于本地项目，这两种运行方式都沿用相同的题号与源程序。

名字属于哪个命名空间

std 是标准库使用的命名空间。std::cout 中的 :: 用来限定名字所属的范围，不是成员访问的小数点。同名变量也可以分别位于 aim 和 sim 命名空间，通过 aim::speed 和 sim::speed 区分。include 与 Python import 在使用形式和语义上不同：这里先把 include 理解为让编译器看到接口，避免把它理解成执行一个 Python 模块。

配套练习

S07 命名空间同名变量

源文件：S07.cpp

aim::speed 为 3，sim::speed 为 8，输出二者，不使用 using namespace。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
无输入	3 8

初始代码

```
#include <iostream>

int main() {
    // TODO
    return 0;
}
```

错误出现在哪一步

漏分号、类型不匹配等问题通常在编译时报告。函数有声明却没有可用定义，可能在链接时报告。程序成功启动之后仍可能发生运行错误，也可能正常结束却算错结果，后者是逻辑错误。先读第一条有意义的诊断，再看它指向的代码位置；一个较早的语法错误可能引发后面许多连带提示。

配套练习

S08 故障分层

源文件：S08.cpp

程序读取一个整数并输出平方。输入保证绝对值不超过 1000。说明缺分号、未定义函数、结果错误各在哪一层发现。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
7	49

初始代码

```
#include <iostream>

int main() {
    // TODO
    return 0;
}
```

1.2 类型、变量与算术

Python 的变量知识可以直接帮助你理解赋值，但 C++ 还要求你明确对象的类型、初始化方式和数值范围。

本节题号：S03、S09、S10、S11、S12、S13、S14、S15、S16

声明、初始化和赋值

在 C++ 中，int count = 0 声明了一个 int 对象，并在创建时把它初始化为 0。后面再写 count = 5 是赋值，它修改一个已经存在的对象。Python 的名字可以重新绑定到不同类型的对象，而这里 count 的类型不会因为一次赋值而改变。这体现 C++ 的静态类型约束。Python 通常采用动态类型，对象具有类型，而名字可以绑定不同类型的对象。类型系统与编译、解释的执行方式是两个比较维度。未初始化的普通局部整数不能拿来直接计算；本课的写法都先给出明确初值。

```
int count = 0;
double speed = 0.0;
bool ready = false;
const int limit = 100;
```

const 表示不能通过这个名字修改对象。它适合表达固定阈值等约束。单引号包围的 char 表达一个字符，例如方向指令；双引号表示字符串字面量。C++ 也有自动推导类型的写法，但这门入门课主要使用明确类型，让你能看出每个表达式正在进行什么计算。

选择足够的表示范围

类型	用途	需要留意的地方
int	常见整数、计数与编号	有固定表示范围
long long	更大范围的有符号整数	适合大整数乘积和总和
unsigned long long	非负大整数	也有上界，不允许无限增长
double	测量值与实数计算	采用有限精度表示
bool / char	真假值 / 单个字符	不等同于任意整数或字符串

标准并不保证所有平台的 int 都恰好占四字节。课程平台常见 int 为 32 位，但程序选择类型时要看题目的范围。两个不超过 10^9 的整数相乘，结果可达到 10^18，需要更大整数类型。long long answer = a * b 只改变最终接收类型，如果 a 和 b 都是 int，中间乘法仍先按 int 计算。可以让操作数提前成为 long long，或在乘法前使用 static_cast<long long>。有符号整数溢出不能作为可靠的“自动绕回”。

输入是解析字符数据

std::cin >> a >> b 依次尝试按变量的类型读取两个值。格式化读取通常先跳过空格或换行，再解析数值，成功后继续从当前位置往后读。输入 12 和 34 时，分隔它们的是空白还是换行，通常不影响整数读取。这里先使用合法输入，下一节再学习读取失败时如何判断。

配套练习

S03 读取两个整数

源文件：S03.cpp

读取两个 [-1000,1000] 整数，输出和与差。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
8 4	12 4

初始代码

```
#include <iostream>

int main() {
    // TODO
    return 0;
}
```

类型决定除法怎样计算

整数除法得到整数商，向零截断。3 / 2 的结果是 1，3.0 / 2 的结果是 1.5。Python 的 / 通常得到实数结果，// 按向下取整规则处理，负数情况下尤其不能把 Python // 和 C++ 整数除法当成同一种规则。% 表示整数余数，拆分秒数或十进制数位时可以结合商与余数使用。

```
double ratio = static_cast<double>(a) / b;
long long product = static_cast<long long>(a) * b;
std::cout << std::fixed << std::setprecision(2) << ratio;
```

上面的 setprecision 需要 <iomanip>。和 fixed 配合时，它控制小数点后位数，格式设置会继续影响同一个流后面的输出。求平均值要让除法发生在浮点类型上；写到 double 变量里的整数商不会重新变回小数。计算摄氏温度换算时，同样要注意 9 / 5 和 9.0 / 5.0 的区别。

配套练习

S09 整数除法与转换

源文件：S09.cpp

读取两个正整数 a、b，分别输出整数商和两位小数的实数商。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
3 2	1 1.50

初始代码

```
#include <iostream>
#include <iomanip>

int main() {
    // TODO
    return 0;
}
```

S10 秒数拆分

源文件：S10.cpp

输入 [0,1000000] 秒数，输出小时、分钟、秒，不补零。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
3661	1 1 1

初始代码

```
#include <iostream>

int main() {
    // TODO
    return 0;
}
```

S11 三次测量平均值

源文件：S11.cpp

输入三个 [-1000,1000] 整数，输出平均值，保留两位小数。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
1 2 3	2.00

初始代码

```
#include <iostream>
#include <iomanip>

int main() {
    // TODO
    return 0;
}
```

S12 大整数乘法

源文件：S12.cpp

输入两个 [0,10^9] 整数，输出乘积。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
1000000000 1000000000	1000000000000000000

初始代码

```
#include <iostream>

int main() {
    // TODO
    return 0;
}
```

S13 摄氏转华氏

源文件：S13.cpp

输入摄氏温度 [-100,200]，按 F=C*9/5+32 输出一位小数。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
25	77.0

初始代码

```
#include <iostream>
#include <iomanip>

int main() {
    // TODO
    return 0;
}
```

数学函数、近似值和向上整除

<cmath> 中的 std::sqrt 用来计算平方根，std::abs 可用于求差的绝对值。浮点值以有限精度表示，比较测量结果时常判断差值是否处在允许误差内，而不是机械地要求完全相等。题目若已经规定 1e-6 的容差，就严格按其绝对误差要求判断。

非负整数 n 放入容量 k 的箱子，k 必须为正。向上整除可以写为 n / k 加上“n % k 是否非零”的补偿项。常见的 (n + k - 1) / k 也需要确认加法不会超出类型范围。选择表达式时，范围分析应覆盖中间步骤。

配套练习

S14 欧氏距离

源文件：S14.cpp

输入点 x、y，输出它到原点的距离，保留两位小数。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
3 4	5.00

初始代码

```
#include <iostream>
#include <iomanip>
#include <cmath>

int main() {
    // TODO
    return 0;
}
```

S15 浮点容差比较

源文件：S15.cpp

输入两个 [-1000,1000] 浮点数，绝对差不超过 1e-6 输出 SAME，否则 DIFF。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
0.3 0.3000001	SAME

初始代码

```
#include <iostream>
#include <cmath>

int main() {
    // TODO
    return 0;
}
```

S16 非负向上整除

源文件：S16.cpp

输入 0≤n≤10^9、1≤k≤10^9，计算至少几个容量 k 的箱子能装下 n 件物品。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
11 5	3

初始代码

```
#include <iostream>

int main() {
    // TODO
    return 0;
}
```

1.3 条件判断与输入状态

条件、分支和逻辑仍然熟悉。需要重新确认的是运算符写法、短路求值，以及流怎样报告一次读取的结果。

本节题号：S17、S18、S19、S06、S20、S21、S22、S23、S24

条件由比较和逻辑组合而成

C++ 的 if 条件写在圆括号里，分支代码放进大括号。Python 的 elif 对应 else if。= 表示赋值，== 表示相等比较。区间判断必须写成 lower <= x && x <= upper，不能照搬 Python 的连续比较写法。比较结果是 bool，可以用 &&、|| 和 ! 分别组合为并且、或者和取反。

```
if (value >= lower && value <= upper) {
    std::cout << "SAFE" << '\n';
} else {
    std::cout << "UNSAFE" << '\n';
}
```

&& 只有在左边为真时才计算右边，|| 只有在左边为假时才计算右边。这种短路求值既能表达规则，也能保护操作。例如先确认分母不为零，再在右侧进行除法。判断“两个条件都成立”时用 &&；只有真的要对二进制位操作时才考虑 &，不能因为外形相近就互换。

配套练习

S17 血量三档

源文件：S17.cpp

输入血量 [0,100]，0 输出 DEAD，1..30 输出 LOW，其余 OK。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
30	LOW

初始代码

```
#include <iostream>

int main() {
    // TODO
    return 0;
}
```

S18 闭区间判断

源文件：S18.cpp

输入速度，位于 [-3,3] 输出 SAFE，其余 UNSAFE。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
-3	SAFE

初始代码

```
#include <iostream>

int main() {
    // TODO
    return 0;
}
```

S19 开火门控

源文件：S19.cpp

输入 hp、heat、enemy（0 或 1），hp>0、heat<100 且 enemy=1 才输出 FIRE，否则 WAIT。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
100 100 1	WAIT

初始代码

```
#include <iostream>

int main() {
    // TODO
    return 0;
}
```

一次输入可能没有得到值

把整数读入 int 时，字符文本可能不符合格式，也可能超出 int 的范围。流会记录这次操作的状态。if (std::cin >> x) 会先读取，再根据流是否处于失败状态决定走哪个分支。它不能保证后面没有多余字符，例如读取 12abc 时整数提取可能先得到 12，把 abc 留在流里；整行是否合法要到字符串解析一节进一步验证。

观察方式	意思
if (stream >> value)	先尝试读取，再检查是否失败
stream.fail()	检查格式失败或底层严重错误状态
stream.eof()	检查是否已经遇到输入序列结尾
std::boolalpha	把布尔输出显示为 true / false

遇到 EOF 与读取失败可以同时发生，但不是同一件事。比如最后一个合法整数恰好读到输入末尾，这次提取仍可成功，同时记录已经到达末尾。后续若再请求一个不存在的整数，读取才会失败。不要用 eof() 提前猜下一次读取是否能成功，应直接检查读取操作。

配套练习

S06 输入失败检测

源文件：S06.cpp

读取一个整数，失败输出 INVALID，成功输出其两倍。范围不超过 1000。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
robot	INVALID

初始代码

```
#include <iostream>

int main() {
    // TODO
    return 0;
}
```

S20 短路保护除法

源文件：S20.cpp

输入 a、b，b 为 0 输出 INVALID，否则输出整数商是否大于 2，使用 boolalpha。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
5 0	INVALID

初始代码

```
#include <iostream>

int main() {
    // TODO
    return 0;
}
```

分支的顺序也是规则的一部分

switch 根据一个整数或字符等离散值选择分支，case 给出匹配值，default 处理其他情况。每个分支通常需要 break，避免继续执行后面的分支。多个条件可能同时满足时，if / else if 的顺序决定采用哪个动作，必须把“优先处理停止”等规则反映在代码结构里。

处理边界时，先把相等情况说清楚：低血量区间是否包含端点，浮点容差是否允许恰好等于阈值，闰年规则里整百和四百的条件如何组合。中位数允许重复时，也要保证相等分支能走到正确结果。分支越复杂，越应该用题目原文逐条检查条件，而不是凭印象写比较符号。

配套练习

S21 switch 指令

源文件：S21.cpp

输入字符 F、L、R 分别输出 FORWARD、LEFT、RIGHT，其他输出 UNKNOWN。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
L	LEFT

初始代码

```
#include <iostream>

int main() {
    // TODO
    return 0;
}
```

S22 三值中位数

源文件：S22.cpp

输入三个 [-1000,1000] 整数，输出中位数，允许重复，不排序。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
9 4 4	4

初始代码

```
#include <iostream>
#include <algorithm>

int main() {
    // TODO
    return 0;
}
```

S23 闰年判断

源文件：S23.cpp

输入 [1,9999] 年份，符合 Gregorian 闰年规则输出 LEAP，否则 COMMON。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
1900	COMMON

初始代码

```
#include <iostream>

int main() {
    // TODO
    return 0;
}
```

S24 安全规则优先级

源文件：S24.cpp

输入 stop、hp、enemy，按 STOP、DEAD、TRACK、PATROL 的顺序选首个适用动作。stop/enemy 为 0 或 1。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
1 0 1	STOP

初始代码

```
#include <iostream>

int main() {
    // TODO
    return 0;
}
```

1.4 循环、EOF 与逐步变化的状态

循环把同一条规则应用很多次。写之前先回答：从什么状态开始，每次更新什么，什么时候停止。

本节题号：S25、S26、S27、S28、S29、S30、S31、S32、S73、S76、S79

for 的执行次序

for (int i = 0; i < n; ++i) 的初始化只执行一次，每轮先检查条件，执行循环体，再更新 i。i < n 对应常见的半开区间 [0,n)，与 Python range(n) 的访问范围一致。++i 让 i 加一。若写成 i <= n，就会多执行一轮；后续索引容器时，这一轮往往越过合法边界。

```
long long total = 0;
for (int i = 0; i < n; ++i) {
    int value = 0;
    std::cin >> value;
    total += value;
}
```

这里每轮只读入一个整数并加入总和。累加器 total 的初值应该代表“还没处理任何元素”的结果。对于求和是 0，对计数也是 0；最大值则不宜随意初始化为 0，因为所有输入可能都为负数。先明确状态含义，再决定初值。

配套练习

S25 循环累加

源文件：S25.cpp

输入 n∈[0,10000]，用 for 计算 1..n 的和。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
5	15

初始代码

```
#include <iostream>

int main() {
    // TODO
    return 0;
}
```

停止条件来自输入或者程序规则

while 每次先判断条件，再执行循环体。while (std::cin >> x) 会尝试读取，成功才进入循环，适合不知道输入个数、一直读到 EOF 的场景。哨兵值则是一条合法读入的数据，但它负责通知程序结束，不一定参与计算。需要在处理数据之前先检查哨兵。

break 立即结束当前所在的一层循环，continue 跳过本轮剩余部分并进入下一轮。for 的 continue 之后仍会执行更新表达式，while 的 continue 则回到条件检查，因此必须避免把必要的读入或更新放在会被跳过的位置。while (!std::cin.eof()) 只能看到过去的状态，可能在循环体里读失败后仍误用旧值。

配套练习

S26 哨兵值输入

源文件：S26.cpp

不断读取整数，遇到 -1 或 EOF 停止，输出之前的和，-1 不参与。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
1 2 3 -1 99	6

初始代码

```
#include <iostream>

int main() {
    // TODO
    return 0;
}
```

S27 过滤与提前停止

源文件：S27.cpp

依次读取整数，负数跳过，0 结束，其余累加。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
3 -2 4 0 9	7

初始代码

```
#include <iostream>

int main() {
    // TODO
    return 0;
}
```

枚举与嵌套循环

整数数位可以通过 % 10 和 / 10 逐次取出并缩短剩余部分。枚举因子时判断 n % d 是否为 0。判断素数需要处理 n < 2 的情况，也要明确只要找到一个合适因子就能提前结束。两层循环可以分别负责行与列，内层完成一行后再换行。这里的重点是控制流，复杂度分析先做到能估计循环次数即可。

配套练习

S28 整数各位和

源文件：S28.cpp

输入 [0,10^9] 整数，输出十进制各位数字之和。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
507	12

初始代码

```
#include <iostream>

int main() {
    // TODO
    return 0;
}
```

S29 因子枚举

源文件：S29.cpp

输入 n∈[1,1000]，按升序每行输出一个正因子。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
6	1 2 3 6

初始代码

```
#include <iostream>

int main() {
    // TODO
    return 0;
}
```

S30 素数判断

源文件：S30.cpp

输入 [0,10^9] 整数，素数输出 YES，否则 NO。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
97	YES

初始代码

```
#include <iostream>

int main() {
    // TODO
    return 0;
}
```

S31 矩形输出

源文件：S31.cpp

输入行数 r 和列数 c，均在 [1,10]，输出 r 行 c 个 #。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
2 3	### ###

初始代码

```
#include <iostream>

int main() {
    // TODO
    return 0;
}
```

状态必须按确定的顺序更新

最长连续长度通常需要两个状态：当前连续长度和历史最大长度。遇到不满足的元素时，只重置当前连续长度，历史最大值继续保留。过滤有效读数时，计数与总和都只在读数有效的分支更新。仿真中“先冷却，再判断，再执行”的顺序会影响结果，不能随意合并或交换。

时刻	需要问自己的问题
进入循环前	各状态代表什么，初值是否正确
处理当前项时	先检查还是先更新，是否应该跳过
本轮结束时	哪些状态保留到下一轮
循环结束后	没有任何输入时是否仍有合理结果

配套练习

S32 连续命中计数

源文件：S32.cpp

输入 n 个 0/1，输出最长连续 1 的长度，n∈[0,1000]。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
7 1 1 0 1 1 1 0	3

初始代码

```
#include <iostream>
#include <algorithm>

int main() {
    // TODO
    return 0;
}
```

S73 有效数据统计

源文件：S73.cpp

输入 n 个读数，只有 [0,100] 有效，输出有效数和有效值总和。n≤1000。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
4 -1 0 100 101	2 100

初始代码

```
#include <iostream>

int main() {
    // TODO
    return 0;
}
```

S76 热量仿真

源文件：S76.cpp

输入 n、初始热量 h、冷却 c、每发热量 d，之后 n 个 0/1。每步先冷却至不低于 0，指令为 1 且 h+d≤100 才射击。输出射击数与最终热量。参数非负≤100，n≤1000。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
3 90 10 20 1 1 1	2 100

初始代码

```
#include <iostream>
#include <algorithm>

int main() {
    // TODO
    return 0;
}
```

定位边界错误

调试循环时可以逐项写出 i 的值，以及本轮读取次数和累加器值。n = 0 时，按次数读取的循环不应再读任何数据。n = 1 时，只处理第一个元素。S79 中的 <= 正是应该检查的边界。把循环范围与数据数量写在纸上，通常比只看最后结果更快发现问题。

配套练习

S79 Debug 循环边界

源文件：S79.cpp

修复求 n 个整数之和的旧代码：for(i=0;i<=n;++i)。输入 n≤1000，逐项读取合法整数。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
3 1 2 3	6

初始代码

```
#include <iostream>

int main() {
    // TODO
    return 0;
}
```

1.5 数组、vector 与一批数据

Python list 的用途在这里仍然存在，但 C++ 会把元素类型和容器的增长方式写进类型或接口。

本节题号：S33、S34、S35、S36、S37、S38、S39、S40、S68、S70、S78

固定长度与动态长度

std::array<int,5> 表示装有五个 int 的固定长度容器，需要 <array>。std::vector<int> 表示可动态改变长度的整数序列，需要 <vector>。尖括号里的内容是类型参数或固定大小参数。现在先学会使用标准库提供的类型，不需要先实现模板。容器名后的点号用来访问它提供的成员操作，例如 values.size()。

```
std::array<int, 5> fixed = {1, 2};
std::vector<int> values;
values.push_back(value);
```

这里 fixed 的其余元素会被零初始化。vector 初始为空，push_back 在末尾追加一项；vector<int> values(n) 则直接创建 n 个整数元素，初值为 0。size() 表示当前元素个数。固定长度和固定容量不是一回事，本课只按题目选择合适容器，不依赖内部如何分配内存。

配套练习

S33 array 初始化

源文件：S33.cpp

创建长度 5 的 `std::array<int,5>`，只显式填入 94、87、50，每行输出一个元素。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
无输入	94 87 50 0 0

初始代码

```
#include <iostream>
#include <array>

int main() {
    // TODO
    return 0;
}
```

S34 数组最大值

源文件：S34.cpp

输入 n≥1 个 [-1000,1000] 整数，输出最大值。n≤1000。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
3 −9 −2 −7	−2

初始代码

```
#include <iostream>
#include <vector>
#include <algorithm>

int main() {
    // TODO
    return 0;
}
```

S35 vector 动态追加

源文件：S35.cpp

读入直到 EOF 的整数，保存到 vector，倒序每行输出。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
1 2 3	3 2 1

初始代码

```
#include <iostream>
#include <vector>
#include <cstddef>

int main() {
    // TODO
    return 0;
}
```

索引要在合法区间内

长度为 n 的序列，其合法索引是 0 到 n-1。C++ 的下标 -1 不表示最后一个元素。array 和 vector 的 operator[] 不提供通常意义上的运行时越界检查，越界访问不能期待统一返回值或稳定报错。需要允许外部输入索引时，先检查 index >= 0，再比较是否小于长度。

size() 返回的是无符号大小类型。把负索引直接与 size() 比较容易受整数转换影响，先检查非负更清楚。本课在题目给出的较小范围内，可以把长度明确转换为 int 后用于有符号循环；不要只为了消除警告就把所有输入索引都改成无符号数。

配套练习

S36 索引合法性

源文件：S36.cpp

固定数组 {10,20,30}，读取有符号索引，合法输出元素，否则 INVALID。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
-1	INVALID

初始代码

```
#include <iostream>
#include <array>

int main() {
    // TODO
    return 0;
}
```

复制整个容器会产生独立的值

Python 中 b = a 通常让两个名字指向同一个列表对象，修改其中一个列表元素会从另一个名字观察到。C++ 中 std::vector<int> b = a 创建一个 vector 副本，两者保存独立的整数元素，修改 b 的元素不会修改 a。这个对照只针对这里的整数容器，不能推广成“任何类型的复制都会递归复制所有外部资源”。引用、指针与对象成员会在后面分别讨论。

操作	a 的状态	b 的状态
复制完成	原来的元素	与 a 内容相同的独立元素
修改 b 的一个元素	保持原值	对应元素改变

配套练习

S37 复制与修改

源文件：S37.cpp

复制 vector {1,2,3} 到 b，修改 b[0]=9，分别输出 a[0] 和 b[0]。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
无输入	1 9

初始代码

```
#include <iostream>
#include <vector>

int main() {
    // TODO
    return 0;
}
```

访问模式比写法更重要

相邻差分访问当前项与前一项，所以从第二个元素开始。过滤时若要求原顺序不变，就按原方向扫描并追加符合条件的项。循环右移可以先用 k % n 去掉整圈移动，避免做没有必要的重复操作；但要先满足 n > 0，才能取余。二维数组把两个索引分别对应到行与列，内层循环通常处理一行中的各列。

配套练习

S38 相邻差分

源文件：S38.cpp

输入 n≥1 个整数，逐行输出后一项减前一项。n≤1000，元素绝对值≤1000。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
3 2 5 3	3 −2

初始代码

```
#include <iostream>
#include <vector>

int main() {
    // TODO
    return 0;
}
```

S39 稳定过滤

源文件：S39.cpp

输入 n 个整数，按原顺序输出非负元素，每行一个。n≤1000。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
4 −2 0 4 −1	0 4

初始代码

```
#include <iostream>
#include <vector>

int main() {
    // TODO
    return 0;
}
```

S40 循环右移

源文件：S40.cpp

输入 n≥1、k≥0 与 n 个整数，右移 k 位，每行输出一个元素。n≤1000，k≤10^9。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
3 4 1 2 3	3 1 2

初始代码

```
#include <iostream>
#include <vector>

int main() {
    // TODO
    return 0;
}
```

S68 二维数组行和

源文件：S68.cpp

读取 2×3 矩阵，逐行输出该行元素和。元素绝对值≤1000。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
1 2 3 4 5 6	6 15

初始代码

```
#include <iostream>
#include <array>

int main() {
    // TODO
    return 0;
}
```

窗口与相邻间隔

长度为 k 的窗口只允许覆盖完整的 k 个元素。当 n >= k 时，窗口起点从 0 到 n-k，一共有 n-k+1 个窗口。均值需要浮点除法。相邻时间戳的差从第二个时间戳开始计算，输入数量为 0 或 1 时都没有相邻间隔。这些题训练的是用索引准确表达数据关系，先保证范围和数值类型正确，再考虑是否需要优化重复计算。

配套练习

S70 固定窗口均值

源文件：S70.cpp

输入 n≥k≥1 和 n 个整数，逐行输出每个长度 k 窗口均值，两位小数。n≤1000，元素绝对值≤1000。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
3 2 1 3 5	2.00 4.00

初始代码

```
#include <iostream>
#include <vector>
#include <iomanip>

int main() {
    // TODO
    return 0;
}
```

S78 丢帧告警

源文件：S78.cpp

输入 n 个递增时间戳，若相邻间隔>10 输出后一个时间戳，每行一个。n≥0，时间戳≤10^9。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
4 0 10 25 26	25

初始代码

```
#include <iostream>

int main() {
    // TODO
    return 0;
}
```

1.6 字符串、整行输入与字符扫描

字符串是一段字符序列。读入方式决定你得到一个词还是一整行，访问方式决定程序怎样逐步处理这些字符。

本节题号：S04、S05、S41、S42、S43、S45、S48、S69、S74、S77

字符和字符串是不同的类型

char 表示单个字符，常用单引号写字面量，例如 'U'。std::string 表示字符串对象，需要 <string>，常用双引号初始化。string 的 size() 返回长度，空字符串长度为 0。先判断字符串非空，才能访问首项与末项。题库中的这些文本处理题使用 ASCII 约束，按字节索引足以覆盖要求，不把同样的处理方式直接推广到中文字符。

与 Python str 不可变的习惯不同，C++ std::string 的字符可以通过合法下标修改。给字符串变量赋一个新值和修改现有字符串中的一个字符是不同操作。这里用现有题目学习最小接口，不介绍 C 风格字符数组和复杂编码转换。

格式化读取与整行读取

std::cin >> name 默认跳过前导空白，并读取到下一个空白之前，适合单词。std::getline(std::cin,name) 读取到换行分隔符，取走该分隔符但不把它保存进 name，适合带空格的一整行。若前面刚用 >> 读完一个数，留在流中的换行会使紧接着的 getline 得到空行。

操作	尚未读取的数据示意
输入内容	数字、换行、下一行名称、换行
提取数字之后	换行、下一行名称、换行
直接 getline	读到第一个换行，得到空字符串
按规则跳过行尾后 getline	得到下一行名称

std::ws 会消耗接下来的所有空白，包括换行和前导空格。只有在题意允许丢弃这些空白时，才使用 getline(std::cin >> std::ws,name)。若需要保留下一行的前导空格，可以先用 ignore 丢弃当前行剩余内容直至换行，再进行 getline；常见写法用 numeric_limits<streamsize>::max() 指定足够大的丢弃上限，需要 <limits>。S05 已限制名称没有前导空格，但读懂这两种做法的差异仍然重要。

配套练习

S04 读取带空格的名称

源文件：S04.cpp

读取一整行机器人名称，输出方括号包围的原文本。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
AIM Sentry 07	[AIM Sentry 07]

初始代码

```
#include <iostream>
#include <string>

int main() {
    // TODO
    return 0;
}
```

S05 数字之后读取整行

源文件：S05.cpp

先读编号，再读下一整行名称。名称允许空格但不含前导空格。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
7 AIM Sentry	7:AIM Sentry

初始代码

```
#include <iostream>
#include <string>

int main() {
    // TODO
    return 0;
}
```

S41 字符串长度与字符

源文件：S41.cpp

输入一行 ASCII 文本，输出长度，非空时再输出首尾字符，空行只输出 0。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
robot	5 r t

初始代码

```
#include <iostream>
#include <string>

int main() {
    // TODO
    return 0;
}
```

查找与修改要先确认命中

find 返回首次命中的位置，未找到时返回 std::string::npos。必须先判断是否为 npos，再拿位置做替换或其他索引操作。replace 可以从某个位置起替换指定数量的字符，替换前后的长度不必相同。处理 ASCII 小写转大写时，只改变 a 到 z 范围内的字符，空格、数字和其他标点按题意保留。

回文检查比较左右对称字符，空串按题意可以直接成立。若用无符号 size() 去计算 size()-1，空串会造成下溢，因此先检查空串或使用半程索引。库接口的返回类型和边界必须一起考虑，不能只记住一个方法名。

配套练习

S42 查找子串

源文件：S42.cpp

输入一行文本，再输入一行关键词，输出首次出现的下标，找不到输出 -1。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
AIM Sentry Sentry	4

初始代码

```
#include <iostream>
#include <string>

int main() {
    // TODO
    return 0;
}
```

S43 安全替换首次命中

源文件：S43.cpp

输入一行文本，把第一次 anon 替换成 saki，不存在则保持原文。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
soyo,anon,taki	soyo,saki,taki

初始代码

```
#include <iostream>
#include <string>

int main() {
    // TODO
    return 0;
}
```

S45 ASCII 小写转大写

源文件：S45.cpp

输入一行 ASCII 文本，仅把 a..z 转为 A..Z。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
aim-07	AIM-07

初始代码

```
#include <iostream>
#include <string>

int main() {
    // TODO
    return 0;
}
```

S48 字符串回文

源文件：S48.cpp

输入一行 ASCII 文本，逐字节判断是否回文，空串算回文，输出 YES 或 NO。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
level	YES

初始代码

```
#include <iostream>
#include <string>
#include <cstddef>

int main() {
    // TODO
    return 0;
}
```

字符扫描怎样形成状态

方向指令通过每个字符修改当前位置。连续字符分组要保存当前字符及计数，遇到变化时完成上一组，并在遍历结束后处理最后一组。空串没有任何分组。带边界的位置更新可以先算候选坐标，再决定接受移动还是保留原位置。这样的“先检查，再提交”与前面解析输入、判断有效数据的结构相通。

配套练习

S69 二维坐标更新

源文件：S69.cpp

输入由 UDLR 组成的单词，从 (0,0) 出发，U 增 y，R 增 x，输出终点。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
UURDLR	1 1

初始代码

```
#include <iostream>
#include <string>

int main() {
    // TODO
    return 0;
}
```

S74 运行长度编码

源文件：S74.cpp

输入一行 ASCII 字符串，把连续相同字符编码为字符:次数，每段一行，空串无输出。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
aaabbc	a:3 b:2 c:1

初始代码

```
#include <iostream>
#include <string>
#include <cstddef>

int main() {
    // TODO
    return 0;
}
```

S77 网格碰撞

源文件：S77.cpp

输入宽 w、高 h 与 UDLR 指令串，从 (0,0) 出发，U 增 y。越界动作原地不动且碰撞加一。w,h≥1，输出 x、y、碰撞数。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
2 2 RRUU	1 1 2

初始代码

```
#include <iostream>
#include <string>

int main() {
    // TODO
    return 0;
}
```

1.7 字符串流与严格解析

输入流可以连接标准输入，也可以连接内存中的字符串。两者都能使用格式化提取，这让“读一整行，再验证字段”成为自然的处理方式。

本节题号：S44、S46、S47、S75

同样的流接口，更换数据来源

<sstream> 提供基于字符串的流。std::istringstream 从一段字符串提取值，std::ostringstream 把值格式化写入字符串，std::stringstream 同时支持读与写。可以把它们看成由字符串缓冲承接数据的流接口。Python 的 io.StringIO 也有把字符串当作流使用的用途，如果你没用过它，直接从本节的读写过程理解即可。

类型	主要方向	常见用途
istringstream	字符串内的数据进入变量	分词、字段解析、读取数值
ostringstream	变量的文本表示进入字符串	组合、格式化一段文本
stringstream	同一字符串缓冲支持输入和输出	需要两种方向的文本流操作

字符串、字符串流对象、流内的缓冲内容是需要区分的概念。初始化流时给出的文本成为其缓冲内容，随后 >> 推进读取位置；读取变量不会自动修改原来的 string 变量。写入输出字符串流后，可以调用 str() 取得当前文本。str() 不表示“读取一个单词”，它访问的是整个缓冲内容。

```
std::istringstream input(line);
int value = 0;
input >> value;

std::ostringstream output;
output << value;
std::string text = output.str();
```

空白分词与指定分隔符

while (input >> word) 按空白提取单词，连续空格通常只作为分隔，不会产生空单词。指定逗号分隔时，空字段却可能有实际意义：开头逗号、相邻逗号和结尾逗号都需要保留相应字段。使用 getline(stream,field,',') 时，要特别考虑结尾分隔符后那个空字段，不能假设一般提取循环会自动补出。

配套练习

S44 逗号字段拆分

源文件：S44.cpp

输入一行，按逗号拆分，每个字段用 [] 包住并逐行输出，保留首尾与相邻空字段。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
a,,b,	[a] [] [b] []

初始代码

```
#include <iostream>
#include <string>
#include <cstddef>

int main() {
    // TODO
    return 0;
}
```

S46 单词计数

源文件：S46.cpp

输入一行，按空白分词，输出单词个数。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
AIM Sentry 07	3

初始代码

```
#include <iostream>
#include <string>
#include <sstream>

int main() {
    // TODO
    return 0;
}
```

读取一个整数与整行合法是两个层次

要确认一整行只有一个整数和允许的前后空白，可以先提取整数，再尝试提取一个额外字符。字符提取默认跳过空白，若还能读到字符，说明尾部存在额外内容；若只剩空白或已经结束，额外字符提取会失败。第一步也必须成功，否则数字格式错误或数值越界都不能接受。S47 的检查就建立在这两个层次上。

读到尾部不意味着任何先前操作都失败。要把每一次提取的返回状态和它对应的任务联系起来：第一次提取应该成功，第二次用于检查是否还有多余内容。流状态会留在流对象里，后续提取也受它影响。

配套练习

S47 严格整数解析

源文件：S47.cpp

读取一行，仅允许前后空白和一个 int，尾部存在额外字符或溢出时输出 INVALID。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
12abc	INVALID

初始代码

```
#include <iostream>
#include <string>
#include <sstream>

int main() {
    // TODO
    return 0;
}
```

缓冲内容、位置与状态分别管理

str(new_text) 设置新的缓冲内容，clear() 默认重置流的错误状态。clear() 的名字容易让人误解为“删除字符串”，实际上它不清空文本，也不负责把读取位置移回开头。复用一个曾失败的流时，通常需要既替换输入文本，又清除之前的失败标记；初学阶段为每一行新建一个 istringstream 往往更容易追踪。

逐行命令处理可以分成两个阶段：先用 getline 取得一条完整文本，再用局部字符串流验证命令和参数。只有整条命令满足规定，才修改累加器或其他业务状态。这样一条坏输入不会让主输入流卡在同一个格式错误上，也不会让一半解析成功的内容提前生效。

配套练习

S75 受限命令解释器

源文件：S75.cpp

输入多行 ADD 整数或 RESET，格式合法才生效，其他忽略。最后输出累加器值，数值绝对值与行数≤1000。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
ADD 5 BAD 9 RESET ADD 3	3

初始代码

```
#include <iostream>
#include <string>
#include <sstream>

int main() {
    // TODO
    return 0;
}
```

1.8 函数、作用域与值传递

函数把计算的输入、输出和局部状态放在一个清楚的边界内。先理解值怎样传进去，再讨论怎样允许函数修改外部对象。

本节题号：S49、S50、S54、S51、S52、S53、S55、S57

函数接口说明什么

C++ 函数在名字之前写返回类型，在参数列表里逐个写参数类型。调用表达式把实参值交给形参，return 把结果交回调用方。没有返回值的函数可以使用 void。与 Python def 相比，显式类型让编译器在调用处检查参数是否符合接口。函数应该完成一件能清楚描述的事，例如限幅或计算一个结果。

```
int add(int a, int b) {
    return a + b;
}
```

调用函数时，编译器需要已经看见合适的声明。可以把整个定义放在 main 之前，也可以先写函数声明，在后面给定义。声明说明接口，定义提供函数体。它们的参数类型和返回类型必须对应。平台练习继续把这些内容保留在一个 .cpp 文件里，不需要先引入多文件构建。

配套练习

S49 返回值与参数

源文件：S49.cpp

读取两个 [-1000,1000] 整数，调用 add 返回和并输出。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
4 5	9

初始代码

```
#include <iostream>

int main() {
    // TODO
    return 0;
}
```

S50 声明与定义

源文件：S50.cpp

在 main 前声明 square，定义也放在程序中，输入 [-1000,1000] 的 x，输出平方。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
5	25

初始代码

```
#include <iostream>

int main() {
    // TODO
    return 0;
}
```

作用域限制一个名字在哪里可见

函数参数和普通局部变量只在相应范围内可见，内层代码块还可以创建自己的局部变量。同名的内层名字会遮蔽外层名字，离开内层块后，外层名字重新可见。作用域主要回答“这里能用哪个名字”，生命周期回答“对象何时存在”，两者常相关但不是完全相同的概念。

配套练习

S54 局部作用域

源文件：S54.cpp

外层 x=1，内部代码块定义 x=2，先输出内层 x 再输出外层 x。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
无输入	2 1

初始代码

```
#include <iostream>

int main() {
    // TODO
    return 0;
}
```

S51 速度限幅函数

源文件：S51.cpp

实现 clamp_speed(v,limit)，输入 v 与非负 limit，输出夹到 [-limit,limit] 的值。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
8 3	3

初始代码

```
#include <iostream>
#include <algorithm>

int main() {
    // TODO
    return 0;
}
```

默认参数、重载和递归

默认参数允许调用方省略某些末尾参数，此时采用声明中指定的默认值。重载允许多个同名函数接收不同参数类型或数量，编译器根据调用选择合适版本，不能只用返回类型区分重载。Python 里相同函数名的重新定义与这里的一组重载不是同一种机制。

递归函数在函数体内再次调用自己。每次调用都有自己的参数和局部状态，因此必须有能够到达的结束条件。阶乘还需要判断结果是否在整数类型范围内。题目已经给出 n 的上限时，应同时把结束条件和返回类型写进设计，不能等计算溢出后再试图纠正。

配套练习

S52 默认参数

源文件：S52.cpp

实现缩放函数 scale(value,factor=2)，读取 value，分别用默认倍数和 3 倍调用。范围≤1000。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
4	8 12

初始代码

```
#include <iostream>

int main() {
    // TODO
    return 0;
}
```

S53 函数重载

源文件：S53.cpp

重载 label(int) 与 label(double)，分别返回 INT 和 DOUBLE，输出 label(3)、label(3.0)。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
无输入	INT DOUBLE

初始代码

```
#include <iostream>
#include <string>

int main() {
    // TODO
    return 0;
}
```

S55 递归阶乘

源文件：S55.cpp

输入 n∈[0,20]，递归计算 n!，返回 unsigned long long。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
5	120

初始代码

```
#include <iostream>

int main() {
    // TODO
    return 0;
}
```

值参数拥有自己的副本

形参 int a 接收的是实参值形成的局部对象，修改 a 不会自动修改调用者的那个整数。交换函数若只交换两个值形参，就只改变了本次调用内部的两个副本。函数返回后，调用方变量仍保持原值。下一节要解决的正是这个明确的问题：如何让函数用另一个名字直接访问原对象。

阶段	调用方对象	值形参
进入函数	保留原值	用实参值初始化局部副本
修改形参	不随之改变	副本发生变化
函数返回	仍保留原值	普通局部副本结束生命周期

配套练习

S57 值传递不能交换实参

源文件：S57.cpp

swap_value 中交换副本，输入 a、b 后调用并输出原实参。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
10 20	10 20

初始代码

```
#include <iostream>

int main() {
    // TODO
    return 0;
}
```

1.9 内存、地址与引用

在写出 & 之前，先建立一个模型：对象占用存储，存储中的内容有类型解释，地址用于标识所在位置。

本节题号：S58、S56、S62

位置与内容需要分开理解

内存可以简化想象为一串有编号的存储单元，字节是观察对象大小时常用的单位。C++ 中一个字节至少能容纳八位，现代常见平台一般为八位。一个 int 往往占用连续的多个字节，sizeof(int) 可以取得当前实现中的大小。地址描述对象所在的位置，值描述对象存储的内容；地址数值与对象里保存的整数不是同一个东西。

下面的地址只用来辅助理解，不是任何题目的固定输出。不同运行、不同机器甚至编译选项都可能改变实际地址。初学时不需要从地址推导真实硬件布局，也不需要假定所有局部变量一定按代码顺序紧邻存放。

对象或名字	示意位置	当前内容
x	0x1000	整数 5
y	0x2000	整数 5
引用 ref	与 x 关联	通过它访问 x

复制值和增加别名

int y = x 会创建另一个整数对象，它从 x 当前的值开始，但之后拥有独立状态。int& ref = x 则让 ref 作为访问 x 的另一个名字。通过 ref 赋值就会修改 x。普通左值引用初始化时需要绑定合适对象，后面的赋值不会让它改绑到另一个对象，而是修改已经绑定的目标。

```
int x = initial;
int copy = x;
int& ref = x;
ref = updated;
```

表达式 &x 取得 x 的地址，而声明 int& ref 中的 & 是引用类型的一部分。把两者写在不同位置看，比把 & 背成一个固定中文词更准确。&ref 与 &x 比较会反映它们关联同一个对象；这不表示标准要求引用必须以某种特定的指针布局实现。

引用参数让函数访问原对象

函数参数改成 int& a 后，调用时 a 绑定调用方提供的整数对象，修改 a 就会影响原对象。交换两个引用参数时，要先保存其中一个旧值，再完成两次赋值，不能在旧值丢失后恢复它。这里的意义是参数机制，与交换算法本身分开理解。

配套练习

S58 引用交换

源文件：S58.cpp

用 int& 实现 swap_ref，读取两个整数并交换后输出。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
10 20	20 10

初始代码

```
#include <iostream>

int main() {
    // TODO
    return 0;
}
```

返回状态与输出参数

一个函数可以返回 bool，表示某次操作是否成功，再通过引用参数交回额外结果。设计接口时需要写清失败时输出参数会怎样：保持原值，还是设置成指定默认值。S56 要求失败时不修改 out，这意味着必须先检查前提，再执行写入。调用方也应该先看返回状态，再把输出值当作有效结果。

配套练习

S56 布尔结果与输出参数

源文件：S56.cpp

实现 safe_divide(a,b,out)，b=0 返回 false 且不修改 out，否则写入整数商。输入范围≤1000。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
5 0	false 99

初始代码

```
#include <iostream>

int main() {
    // TODO
    return 0;
}
```

只读借用避免无意复制

const std::vector<int>& values 接收一个容器的只读引用，不需要为这次参数传递复制整组元素，也不允许通过这个引用修改元素。const 限制的是这条访问路径，不意味着对象在所有其他路径上都绝对不会变化。只读遍历时可以写 const int& item，也可以对很小的整数按值读取；较大对象通常更需要注意复制成本。

Python 的名字绑定模型在这里是一个有用对照：两个名字可以关联同一个对象，但 C++ 对象默认按值复制、引用绑定后不改绑等规则必须分别学习。不能把所有赋值都想成 Python 名字重新指向对象。

范围 for 的冒号左侧声明本轮访问元素的方式，右侧给出要遍历的容器。写成 const int& item 时，item 在每一轮借用对应元素，循环体只能通过它读取。把引用去掉则按值取得本轮元素副本。冒号这里是范围遍历语法，与构造函数的成员初始化列表没有关系。

```
for (const int& item : values) {
    // read item
}
```

配套练习

S62 const 引用遍历

源文件：S62.cpp

用 const vector<int>& 实现 sum，不复制 vector，输入 n 个≤1000 的整数，输出和。n≤1000。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
3 1 2 3	6

初始代码

```
#include <iostream>
#include <vector>

int main() {
    // TODO
    return 0;
}
```

1.10 指针、区间与对象有效期

引用提供别名，指针则把地址作为一个可以保存和修改的值。只有同时理解“指向哪里”和“目标还活着吗”，才能安全使用它。

本节题号：S59、S60、S61、S64、S63

指针变量与目标对象

int* p 声明一个指向 int 的指针对象。int* p = &x 把 x 的地址保存到 p。p 本身也占用存储，保存的内容是地址值；x 是另一个对象，保存整数值。表达式 *p 通过 p 保存的地址访问目标对象，这个操作叫解引用。声明中的星号、解引用表达式中的星号和乘法运算符需要按位置区分。

```
int* p = &x;
*p = updated;
p = &other;
```

语句	改变谁	保持什么
*p = updated	当前指向的目标对象	p 保存的地址
p = &other	指针对象 p	原目标对象的内容

一个普通指针可以重新保存另一个地址，引用则保持原来的绑定关系。指针复制只复制地址值，两份指针可以访问同一个目标，并不会复制目标对象。这也说明“值复制”不等于所有可达资源都深复制。

配套练习

S59 指针读写

源文件：S59.cpp

x=5，用指针指向它并将其改成 9，输出 x 以及指针是否等于 &x。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
无输入	9 true

初始代码

```
#include <iostream>

int main() {
    // TODO
    return 0;
}
```

空指针与只读指针

nullptr 是空指针值，表示这里没有可访问的目标。未初始化的指针不会自动等于 nullptr，必须先初始化。使用 *p 前要保证 p 指向一个仍然有效、类型合适的对象；检查 p 非空只满足其中一个条件，不能证明对象仍然存活。

const int* p 表示不能通过 p 修改所指整数，但 p 可以改指向。int* const p 表示指针自身固定，目标是否可写则由所指类型决定。题库主要使用第一种，即只读借用的指针。把 const 修饰对象还是指针分开读，就不必依赖容易混淆的口号。

配套练习

S60 空指针保护

源文件：S60.cpp

safe_increment 接收可能为空的 int*，非空时加一。依次用 nullptr 和 &x（x=7）调用，输出 x。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
无输入	8

初始代码

```
#include <iostream>

int main() {
    // TODO
    return 0;
}
```

S61 指向 const 的指针

源文件：S61.cpp

a=3、b=7，const int* p 先指向 a 后指向 b，逐行输出解引用值。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
无输入	3 7

初始代码

```
#include <iostream>

int main() {
    // TODO
    return 0;
}
```

地址本身不携带区间长度

把内建数组传给接受指针的参数时，常会转换为首元素指针。指针并不会自动告诉函数后面还有多少个元素，因此指针区间接口通常同时接收 p 和 n。只有 i 在 [0,n) 且这些元素确实存在时，p[i] 才有意义。本课的 sum_span 是普通自定义函数名，不是 C++20 的 std::span。

约定 n = 0 时允许 p = nullptr，意味着函数在空区间内不得解引用 p，也不能为了“试试看”先访问 p[0]。非空时，调用方负责提供足够长且存活的连续区间。标准容器通常更容易携带长度信息，因此实际接口应优先考虑能明确表达范围的形式。

配套练习

S64 非拥有指针区间

源文件：S64.cpp

sum_span(p,n) 对 n 个元素求和，n=0 时允许 p=nullptr；演示数组与空区间。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
无输入	12 0

初始代码

```
#include <iostream>
#include <cstddef>

int main() {
    // TODO
    return 0;
}
```

函数返回后，哪些对象仍然存在

普通局部对象在相应作用域结束时销毁。返回它的地址或引用，并不会延长它的生命；调用方拿到的访问入口会悬空，继续使用它属于未定义行为，不能靠“某次运行还能读出旧值”证明安全。返回新创建的数据时，按值返回通常是自然选择。

return values 可以把一个 vector 结果交给调用方。实现可能通过复制消除或移动等方式避免不必要的完整复制，初学时先掌握语义：调用方得到一个有效的结果对象，不再借用函数里已经结束的局部对象。第二课会进一步讨论对象的构造、析构与独占所有权。

配套练习

S63 按值返回避免悬空

源文件：S63.cpp

make_values 在函数内部创建 vector{1,2,3} 并按值返回，调用方输出 size 和首元素。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
无输入	3 1

初始代码

```
#include <iostream>
#include <vector>

int main() {
    // TODO
    return 0;
}
```

1.11 结构体、枚举与记录集合

到这里你能处理数值、文本和序列。最后把相关字段组织成明确的记录，并用函数和容器处理多条记录。

本节题号：S65、S67、S66、S71、S72、S80

结构体把相关字段放在一起

一条遥测记录可能同时包含编号、血量和速度。把它们分别保存在互不关联的数组里容易错位，struct 可以把字段组织成一个类型。定义末尾需要分号，成员用点号访问。与 Python 字典相比，结构体在类型定义中明确写出字段名和字段类型，编译器据此检查访问。

```
struct Frame {
    int id = 0;
    int hp = 0;
    double speed = 0.0;
};
```

Frame value 创建该类型的一个对象。vector<Frame> 则保存一批同类型记录。通过 Frame& 遍历时可以把输入直接写入每条记录，通过 const Frame& 遍历时表达只读访问。这正好使用前两节建立的引用概念。结构体还可以有成员函数，下一课再从这里发展到类和封装。

配套练习

S65 结构体遥测帧

源文件：S65.cpp

定义 Frame{id,hp,speed}，读取三个字段后输出 id:hp:speed，speed 保留一位。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
7 80 1.5	7:80:1.5

初始代码

```
#include <iostream>
#include <iomanip>

int main() {
    // TODO
    return 0;
}
```

用枚举表达有限状态

enum class 可以为有限的选项起名字，让方向等状态比裸整数更容易理解。枚举值通过类型名限定，例如 Direction::UP。作用域枚举不会随意隐式转换成 int；需要按数字输入输出时，使用显式转换，并先按题目约束确认编号有效。转向更新还可以结合取余表达循环状态。

```
enum class Direction {
    UP, RIGHT, DOWN, LEFT
};
```

配套练习

S67 枚举方向

源文件：S67.cpp

用 enum class 表示四方向 UP=0、RIGHT=1、DOWN=2、LEFT=3，输入 0..3，右转后输出新编号。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
3	0

初始代码

```
#include <iostream>

int main() {
    // TODO
    return 0;
}
```

比较规则要完整处理平局

最近目标可以比较到原点距离的平方，避免在只比较大小时额外开平方。若距离相同，还要按编号比较，这个平局规则必须写成第二层判断。排序也是同样的思想：比较器返回“第一个对象应排在第二个前面”是否成立。相等对象不能互相都被判定在对方前面。

std::sort 来自 <algorithm>，begin() 与 end() 表达待排序的半开范围。可以把已学的普通函数作为比较器；也可以读懂现有参考实现里的简短 lambda。[] 是捕获列表，参数仍然按普通函数的方式声明，函数体返回 bool。本组比较器只需要读取参数，可用 const 引用，不必引出完整的 lambda 或迭代器专题。

```
bool before(const Record& a, const Record& b) {
    if (a.score != b.score) {
        return a.score > b.score;
    }
    return a.id < b.id;
}
```

配套练习

S66 最近目标

源文件：S66.cpp

输入 n≥1 个 id、x、y，取距离原点平方最小者，相同取 id 小者。坐标绝对值≤1000，n≤1000。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
3 9 3 4 2 -3 -4 7 6 0	2

初始代码

```
#include <iostream>
#include <limits>

int main() {
    // TODO
    return 0;
}
```

S72 排序和平局

源文件：S72.cpp

输入 n 条 id、score，按 score 降序、id 升序输出 id，每行一个。n≤1000。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
3 9 10 7 20 2 20	2 7 9

初始代码

```
#include <iostream>
#include <vector>
#include <algorithm>

int main() {
    // TODO
    return 0;
}
```

筛选、去重与报表

时间戳范围有限时，可以用按时间戳索引的标记记录是否已见过；只在第一次出现时接受记录，就能保持首次记录及输入顺序。不要把“忽略重复”误写成只保留最后一次。范围标记是否可以使用，要由题目给出的时间戳范围决定。

汇总任务把此前的多个小步骤组合起来：读取记录、按规则约束字段、按指定关键字排序、逐条输出，再计算汇总项。各步骤之间要保留必要数据，输出格式也应和题面一致。第一课的目标是能独立读懂这些组合，第二课将在相同的数据与函数基础上，让对象自己维护状态和使用规则。

配套练习

S71 时间戳去重

源文件：S71.cpp

输入 n 条 ts、value，时间戳范围 [0,100]。每个时间戳仅保留首次记录，按输入顺序输出。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
3 2 10 2 99 1 7	2 10 1 7

初始代码

```
#include <iostream>
#include <array>

int main() {
    // TODO
    return 0;
}
```

S80 任务汇总报表

源文件：S80.cpp

输入 n 条 id、hp，hp 限幅到 [0,100]，按 id 升序输出 id:hp，最后输出 ALIVE=存活数。id 唯一，n≤1000。

输入说明

输入约束见题面；无输入题直接运行。

输出说明

严格按题目指定内容与顺序输出。

样例输入	样例输出
2 7 120 2 -5	2:0 7:100 ALIVE=1

初始代码

```
#include <iostream>
#include <vector>
#include <algorithm>

int main() {
    // TODO
    return 0;
}
```

概念资料

本书按 C++17 使用现有题目。以下资料用于核对基本语言与库概念，工作草案网页中的后续标准新增功能不属于本课要求。

Python 官方文档：Execution model
https://docs.python.org/3/reference/executionmodel.html

Python 官方文档：Glossary
https://docs.python.org/3/glossary.html

GCC 官方文档：Overall Options
https://gcc.gnu.org/onlinedocs/gcc/Overall-Options.html

C++ 工作草案：Input/output library
https://eel.is/c++draft/input.output

C++ 工作草案：String-based streams
https://eel.is/c++draft/string.streams

C++ 工作草案：References
https://eel.is/c++draft/dcl.ref

C++ 工作草案：Object lifetime
https://eel.is/c++draft/basic.life

C++ Core Guidelines：Resource management
https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#S-resource

CMake 官方文档：命令行参考
https://cmake.org/cmake/help/v4.0/manual/cmake.1.html

CMake 官方教程：最小项目
https://cmake.org/cmake/help/v4.0/guide/tutorial/A%20Basic%20Starting%20Point.html